

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL 2- Debugging & Optimisation Task
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	Brasilla R	Reg.No.:	192511357
Department:	BTECH CSE	Date:	04.08.2026

ANNEXURE A

1. A signed binary addition program is producing incorrect results when adding negative numbers using 2's complement representation. Debug the logic to identify where overflow or sign handling fails, and optimize the implementation to correctly detect overflow conditions while maintaining efficient execution.
2. A carry look-ahead adder circuit designed for fast addition is not achieving the expected speed improvement over ripple carry addition. Analyze the design to locate delays in generate and propagate logic, and propose optimizations to improve performance without increasing hardware complexity significantly.
3. A multiplication module based on Booth's algorithm is generating incorrect outputs for certain signed inputs. Debug the step-by-step execution of the algorithm to identify errors in bit shifting or decision rules, and optimize the implementation to reduce the number of required cycles.
4. A floating-point arithmetic unit implementing IEEE 754 single precision is producing rounding errors and incorrect normalization. Identify the faults in exponent alignment, mantissa calculation, or rounding logic, and optimize the design to ensure accurate and efficient floating-point operations.
5. A processor's instruction execution pipeline (fetch, decode, execute) is experiencing delays due to improper instruction sequencing and hazards. Debug the pipeline stages to detect data or control hazards, and propose optimization techniques such as pipelining improvements or instruction reordering to enhance overall processor performance.

Code of Conduct:

I Brasilla R(192511357) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R. Bindu

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. Debugging Signed Binary Addition Using 2's Complement

Problem

A signed binary addition program produces incorrect results while adding negative numbers using **2's complement** representation.

Analysis

Possible reasons include:

- Incorrect conversion to 2's complement.
- Sign bit not extended correctly.
- Overflow detection logic is missing.
- Incorrect carry handling.

Example:

```
0100 (+4)
+ 0101 (+5)
-----
1001 (-7) ← Overflow
```

The result is incorrect because two positive numbers produced a negative result.

Overflow Detection

Overflow occurs when:

- Positive + Positive → Negative
- Negative + Negative → Positive

Formula:

Overflow = Carry into MSB XOR Carry out of MSB

OR

If $\text{Sign}(A) = \text{Sign}(B)$

AND $\text{Sign}(\text{Result}) \neq \text{Sign}(A)$

Overflow = TRUE

Optimization

- Use proper 2's complement conversion.
- Perform sign extension before addition.
- Detect overflow using sign-bit comparison.
- Ignore overflow for unsigned operations.

- Use efficient ALU hardware for parallel addition.

Result

The corrected implementation accurately adds signed numbers and reliably detects overflow while maintaining efficient execution.

2. Carry Look-Ahead Adder Performance Analysis

Problem

Carry Look-Ahead Adder (CLA) is not significantly faster than Ripple Carry Adder.

Analysis

Possible delays:

- Slow Generate (G) logic

$$G_i = A_i \times B_i$$

- Slow Propagate (P) logic

$$P_i = A_i \text{ XOR } B_i$$

- Long logic paths
- High fan-out
- Poor gate arrangement

Optimization

Improve Generate/Propagate Logic

Use parallel computation.

Hierarchical CLA

Divide adder into blocks.

Example:

4-bit CLA

↓

16-bit CLA

↓

32-bit CLA

Reduce Fan-Out

Use buffering.

Parallel Prefix Adders

Examples:

- Kogge-Stone
- Brent-Kung

These improve speed with moderate hardware increase.

Result

The optimized CLA reduces carry delay and improves addition speed without significantly increasing hardware complexity.

3. Booth's Multiplication Algorithm Debugging

Problem

Booth's algorithm generates incorrect results for signed multiplication.

Analysis

Common errors:

Incorrect Arithmetic Right Shift

Should preserve sign bit.

Wrong:

0101 $\rightarrow$ 0010

Correct:

1101 $\rightarrow$ 1110

Incorrect Decision Table

Q0 Q-1 Operation

0 0 No Operation

1 1 No Operation

0 1 Add Multiplicand

1 0 Subtract Multiplicand

Incorrect Cycle Count

Algorithm must execute exactly **n** iterations.

Optimization

- Preserve sign during arithmetic shift.
- Implement correct Booth decision logic.
- Skip unnecessary additions.
- Combine shift and add operations.

Result

The optimized Booth multiplier produces correct signed multiplication with fewer arithmetic operations.

4. IEEE 754 Floating Point Unit Debugging

Problem

Floating-point unit produces rounding and normalization errors.

Analysis

Exponent Alignment Error

Before addition:

$$1.101 \times 2^5$$

(+)

$$1.010 \times 2^3$$

Smaller exponent must be shifted.

Mantissa Calculation Error

Perform addition/subtraction only after exponent alignment.

Normalization Error

If result is

$$10.110$$

Normalize to

$$1.0110 \times 2^1$$

Increase exponent.

Rounding Error

IEEE-754 uses

- Guard Bit
- Round Bit
- Sticky Bit

Incorrect implementation causes precision loss.

Optimization

- Correct exponent comparison.
- Normalize after every operation.
- Implement Round-to-Nearest-Even.
- Optimize mantissa alignment hardware.

Result

Floating-point operations become IEEE-754 compliant with accurate rounding and normalization.

5. Pipeline Hazard Debugging

Problem

Processor pipeline suffers delays because of hazards.

Pipeline:

Fetch

↓

Decode

↓

Execute

Types of Hazards

1. Data Hazard

Example

ADD R1,R2,R3

SUB R4,R1,R5

SUB requires updated R1.

Solution

Data Forwarding

ALU Output

↓

Next Instruction

Avoids waiting.

2. Control Hazard

Occurs in branch instructions.

Example

BEQ R1,R2,LABEL

Processor doesn't know next instruction.

Solution

Branch Prediction

Predict branch before completion.

3. Structural Hazard

Two instructions require same hardware simultaneously.

Solution

Duplicate hardware resources.

Optimization Techniques

- Instruction Reordering
- Pipeline Forwarding
- Branch Prediction
- Hazard Detection Unit
- Compiler Scheduling
- Pipeline Balancing

Performance Improvement

Without Optimization

Fetch



Decode



STALL



Execute



STALL



With Optimization

Fetch



Decode



Execute



Memory



Write Back

Continuous execution with minimal stalls.

Result

The optimized pipeline reduces data and control hazards, minimizes stalls, improves instruction throughput, and enhances overall processor performance.